LedgerGuard

Finance Data Processing & Access Control Backend

Assignment Submission Document

Project Name	LedgerGuard
Author	Utkarsh Raj
Email	theutkarshraj@gmail.com
GitHub Repository	https://github.com/theUtkarshRaj/LedgerGuard
Live Backend (API)	https://ledgerguard.onrender.com
Live Frontend	https://ledger-guard-three.vercel.app
Tech Stack	Node.js, Express, PostgreSQL (Neon), Prisma, JWT, Zod
Submitted On	April 2025

1. Project Overview

LedgerGuard is a fully functional REST API backend built for a finance dashboard system. It is designed around the principle of role-based access control, where users with different roles (Viewer, Analyst, Admin) interact with financial data according to well-defined permissions. The backend is live and deployed on Render, connected to a managed PostgreSQL database on Neon.

The project was built from scratch during this assignment, covering every requirement listed: user and role management, financial records CRUD, dashboard summary analytics, access control middleware, Zod-based input validation, JWT authentication, pagination, soft deletes, and comprehensive error handling.

> **Live Deployment**
> Backend API: https://ledgerguard.onrender.com Frontend (separate): https://ledger-guard-three.vercel.app GitHub: https://github.com/theUtkarshRaj/LedgerGuard

2. Architecture & Project Structure

LedgerGuard follows a clean layered architecture separating routes, controllers, services, validators, and middleware. Each layer has a single responsibility, making the codebase easy to navigate and extend.

2.1 Directory Structure

```
backend/
  src/
    routes/          → Express router definitions
    controllers/     → HTTP request/response handlers
    services/        → Business logic and DB access (Prisma)
    middleware/      → Auth guard, role restriction, error handler
    validators/      → Zod schemas for request validation
    utils/           → AppError, asyncHandler helpers
    app.js           → Express app setup, CORS, global middleware
    server.js        → HTTP server entry point
  prisma/
    schema.prisma    → Data models (User, FinancialRecord)
  .env.example       → Environment variable template
```

2.2 Layer Responsibilities

Layer	Responsibility
Routes	Maps HTTP methods and paths to controller functions; applies middleware chains
Controllers	Validates path params, calls service methods, returns HTTP responses
Services	Contains all business logic, database queries via Prisma, data transformations
Middleware	JWT verification (protect), role enforcement (restrictTo), global error handler
Validators	Zod schemas; called inside controllers using schema.parse()
Utils	AppError (structured errors), asyncHandler (promise wrapper for Express)

3. Data Modeling (Prisma + PostgreSQL)

The database schema uses Prisma ORM with a PostgreSQL database hosted on Neon. Two primary models power the system: User and FinancialRecord.

3.1 User Model

```
model User {
  id        Int       @id @default(autoincrement())
  name      String
  email     String    @unique
  password  String    // bcrypt hash
  role      Role      @default(VIEWER)  // VIEWER | ANALYST | ADMIN
  status    Status    @default(ACTIVE)  // ACTIVE | INACTIVE
  createdAt DateTime  @default(now())
  updatedAt DateTime  @updatedAt
  records   FinancialRecord[]
}
```

3.2 FinancialRecord Model

```
model FinancialRecord {
  id         Int       @id @default(autoincrement())
  userId     Int
  amount     Decimal   @db.Decimal(18, 2)
  type       RecordType  // INCOME | EXPENSE
  category   String
  date       DateTime
  note       String?
  deletedAt  DateTime?   // soft delete support
  createdAt  DateTime  @default(now())
  updatedAt  DateTime  @updatedAt
  user       User      @relation(fields: [userId], references: [id])
}
```

3.3 Design Decisions

- Decimal(18,2) for amount ensures financial precision without floating-point errors
- Soft deletes via deletedAt allow data recovery and audit trails
- role and status use Prisma enums for type safety
- Passwords are hashed with bcrypt before storage; never returned in API responses
- User-Record relation enables per-user record scoping and admin-level visibility

4. Authentication & Access Control

4.1 JWT Authentication Flow

Every protected route passes through the protect middleware. It reads the Authorization: Bearer <token> header, verifies the JWT signature, looks up the user in the database, and attaches req.user. If any step fails, a 401 Unauthorized error is returned immediately.

```
// auth.middleware.js (protect)
exports.protect = asyncHandler(async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) throw new AppError('Not authenticated', 401);
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  const user = await prisma.user.findUnique({ where: { id: decoded.id } });
  if (!user || user.status === 'INACTIVE') throw new AppError('Access denied', 401);
  req.user = user;
  next();
});
```

4.2 Role-Based Access Control

After authentication, the restrictTo middleware enforces role-based permissions. It accepts a list of allowed roles and compares against req.user.role. If the user's role is not in the allowed list, a 403 Forbidden is returned.

```
// auth.middleware.js (restrictTo)
exports.restrictTo = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return next(new AppError('You do not have permission', 403));
  }
  next();
};
```

4.3 Role Permission Matrix

Action	VIEWER	ANALYST	ADMIN
View own financial records	✓	✓	✓
Access dashboard summaries	✓	✓	✓
View analytics / insights	✗	✓	✓
Create financial records	✗	✗	✓
Update financial records	✗	✗	✓
Delete financial records	✗	✗	✓

Manage users (list/create/update)	✗	✗	✓
Change user roles & status	✗	✗	✓

5. API Reference

5.1 Authentication Routes /api/auth

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register a new user (default role: VIEWER)
POST	/api/auth/login	None	Login with email & password; returns JWT token
GET	/api/auth/me	JWT	Returns the currently authenticated user's profile

5.2 Financial Records /api/records

Method	Endpoint	Role	Description
GET	/api/records	All	List records (own or all for ADMIN) with filters & pagination
POST	/api/records	ADMIN	Create a new financial record
GET	/api/records/:id	All	Get a single record by ID
PATCH	/api/records/:id	ADMIN	Update fields of an existing record
DELETE	/api/records/:id	ADMIN	Soft-delete a record (sets deletedAt timestamp)

Query Parameters for GET /api/records:

- page, limit — pagination (default: page=1, limit=20, max limit=20)
- from, to — filter by date range (ISO date strings)
- category — filter by category string
- type — filter by INCOME or EXPENSE
- q — full-text search across category and note fields

5.3 Dashboard & Analytics /api/dashboard

Method	Endpoint	Role	Returns
GET	/api/dashboard/summary	All	Total income, expenses, net balance, record count
GET	/api/dashboard/category-breakdown	ANALYST+	Totals grouped by category (income & expense per category)
GET	/api/dashboard/monthly-trend	ANALYST+	Month-by-month income/expense aggregation
GET	/api/dashboard/recent	All	The 10 most recent financial records

5.4 User Management /api/users (ADMIN only)

Method	Endpoint	Role	Description
GET	/api/users	ADMIN	List all users with pagination
POST	/api/users	ADMIN	Create a new user with specified role
GET	/api/users/:id	ADMIN	Retrieve a specific user by ID
PATCH	/api/users/:id	ADMIN	Update user name, role, or status

6. Validation & Error Handling

6.1 Zod Validation

All incoming request bodies and query parameters are validated using Zod schemas defined in the validators/ directory. This provides strong typing, coercion, and descriptive error messages without manual if-else chains.

```
// validators/record.validator.js (excerpt)
const createRecordSchema = z.object({
  userId:   userIdField.optional(),
  amount:   z.union([z.number().positive(), z.string().regex(/^\d+(\.\d{1,2})?$/)]),
  type:     z.enum(['INCOME', 'EXPENSE']),
  category: z.string().trim().min(1).max(80),
  date:     z.coerce.date(),
  note:     z.string().trim().max(2000).optional(),
});
```

Key validation features:

- Amount accepts both number and decimal string to prevent NaN from coerce on empty input
- Dates are coerced automatically from ISO strings
- Enum fields (type, role, status) reject unknown values with a clear error

- Query schemas validate pagination params with min/max bounds
- updateRecordSchema uses .refine() to ensure at least one field is provided

6.2 Error Response Format

All errors flow through a centralized error handler middleware. Zod validation errors are caught and re-formatted into a structured JSON response:

```
// Validation error response:
{
  "status": "fail",
  "message": "Validation error",
  "errors": [
    { "field": "amount", "message": "Amount must be a positive decimal" },
    { "field": "type",   "message": "Invalid enum value" }
  ]
}
```

```
// AppError (operational) response:
{
  "status": "fail",
  "message": "You do not have permission to perform this action"
}
```

6.3 HTTP Status Code Usage

Code	When Used
200 OK	Successful GET, PATCH operations
201 Created	Successful POST (new resource created)
400 Bad Request	Validation errors, invalid query parameters, invalid IDs
401 Unauthorized	Missing or invalid JWT, INACTIVE user account
403 Forbidden	Authenticated but insufficient role
404 Not Found	Resource does not exist or was soft-deleted
500 Internal Server Error	Unexpected server errors (logged; generic message returned)

7. Additional Features Implemented

7.1 Pagination

All list endpoints (records, users) support cursor-style offset pagination with page and limit query parameters. Responses include a meta object with total count, current page, page size, and total pages.

```
// Meta object in list responses:
{
  "data": [...],
  "meta": {
    "total": 84,
    "page": 2,
    "limit": 20,
    "totalPages": 5
  }
}
```

7.2 Soft Deletes

Deleted financial records are not removed from the database. Instead, a deletedAt timestamp is set. All queries automatically filter out soft-deleted records using Prisma's where: { deletedAt: null } condition, ensuring the data is recoverable and the audit trail is preserved.

7.3 Full-Text Search

The GET /api/records endpoint supports a q query parameter that performs a case-insensitive search across the category and note fields using Prisma's contains filter. This allows users to quickly find specific records without knowing the exact category.

7.4 CORS Multi-Origin Support

The backend supports multiple allowed origins via a dynamic CORS configuration, permitting both the production Vercel frontend and local development origins simultaneously. This eliminates the need to change environment variables between environments.

7.5 Environment Configuration

A .env.example file is committed to the repository documenting all required environment variables:

- PORT — HTTP server port
- DATABASE_URL — PostgreSQL connection string (Neon)
- JWT_SECRET — Secret key for signing tokens

- JWT_EXPIRES_IN — Token expiry duration
- ALLOWED_ORIGINS — Comma-separated list of permitted CORS origins

7.6 Integration Test Runner

A lightweight test script (public.test.js extracted from git objects) validates core public endpoints: the root health check, login rejection with bad credentials, protected route enforcement without a token, and user registration. This demonstrates testability of the API design.

8. Assignment Requirements Mapping

The table below maps each core requirement from the assignment to its implementation in LedgerGuard.

Requirement	Implementation in LedgerGuard
User & Role Management	User model with VIEWER/ANALYST/ADMIN roles and ACTIVE/INACTIVE status; admin-only CRUD via /api/users
Financial Records CRUD	Full CRUD on /api/records with Prisma; ADMIN-only write access; all roles can read
Dashboard Summary APIs	/api/dashboard/* with 4 endpoints: summary, category-breakdown, monthly-trend, recent
Access Control Logic	protect middleware (JWT) + restrictTo middleware (roles) applied at route level
Input Validation	Zod schemas for all request bodies and query params; centralized error formatting
Error Handling	AppError class + asyncHandler + global error middleware with appropriate status codes
Data Persistence	PostgreSQL on Neon via Prisma ORM; schema migration with prisma migrate
Authentication (Optional)	JWT-based auth with bcrypt password hashing; /api/auth/register, /login, /me
Pagination (Optional)	page + limit on all list endpoints with meta response
Search (Optional)	q param on /api/records for full-text search on category + note
Soft Deletes (Optional)	deletedAt field on FinancialRecord; auto-filtered in all queries
API Documentation (Optional)	README with endpoint reference, example requests/responses, setup guide

9. Assumptions & Tradeoffs

Assumptions

- Roles are flat (not hierarchical). ADMIN includes ANALYST and VIEWER permissions by explicit route configuration rather than role inheritance.
- Financial records belong to a user via userId. Admins can create records on behalf of any user; non-admins see only their own records.
- Amounts are stored as Decimal(18,2). Currency is not modeled (single-currency system assumed).
- Registration always assigns VIEWER role. Role changes require Admin action.
- Soft-deleted records are invisible to all API consumers. No undelete endpoint was built (out of scope).

Tradeoffs

- SQLite vs PostgreSQL: PostgreSQL (Neon) was chosen over SQLite to demonstrate production-readiness and to support the live deployment requirement.
- Simple JWT vs sessions: Stateless JWT was chosen for its simplicity and compatibility with the Vercel-hosted frontend without shared session storage.
- No rate limiting implemented: Out of scope for the assignment, but the structure supports easy integration of express-rate-limit middleware.
- No unit tests: A lightweight integration test runner was built instead, which better demonstrates API correctness at the boundary level.

10. Local Setup Guide

Prerequisites

- Node.js v18+
- PostgreSQL database (or Neon free tier)
- npm or yarn

Steps

```
# 1. Clone the repository
git clone https://github.com/theUtkarshRaj/LedgerGuard.git
cd LedgerGuard/backend

# 2. Install dependencies
npm install

# 3. Configure environment
```

```
cp .env.example .env
# Edit .env: set DATABASE_URL, JWT_SECRET, etc.

# 4. Run database migrations
npx prisma migrate deploy

# 5. Start the server
npm start          # production
npm run dev        # development (nodemon)
```

> **Live API**
> The API is already deployed and accessible at: https://ledgerguard.onrender.com Example: GET https://ledgerguard.onrender.com/api/auth/me (with Authorization header)

Thank you for reviewing LedgerGuard

Built with care by Utkarsh Raj — theutkarshraj@gmail.com

github.com/theUtkarshRaj/LedgerGuard